

C# Karar Yapılarına Giriş

Görsel Programlama - Hafta 8

C# Karar Yapılarına Giriş: Program Akışının Kontrolü

Programlamada akış kontrolü (Control Flow), komutların hangi sırayla çalıştırılacağını belirler.

Karar yapıları, belirli bir koşulun (boolean ifade) sonucuna göre farklı kod bloklarının yürütülmesini sağlar.

Bu durum, programların dinamik ve girdi odaklı çalışabilmesi için hayati önem taşır.

Üniversite düzeyinde, algoritmik verimlilik ve okunabilirlik açısından bu yapıların doğru kullanımı kritik öneme sahiptir.



Koşullu Yürütmenin Temelleri

Karar yapıları, yalnızca 'doğru' (true) veya 'yanlış' (false) olarak değerlendirilebilen Boolean koşullara dayanır.

Koşul doğruysa bir kod yolu, yanlışsa (genellikle else ile) başka bir kod yolu izlenir.

Bu mekanizma, gerçek hayattaki karar verme süreçlerini dijital ortama taşır.



C# Dilinde Temel Anahtar Kelimeler

C# dilinde temel karar yapısı, if ve else anahtar kelimeleriyle kurulur.

if (koşul): Belirtilen koşulun doğru (true) olması durumunda çalışacak kod bloğunu tanımlar.

else: if koşulunun yanlış olması durumunda alternatif olarak çalışacak kod bloğunu tanımlar.

Bu temel yapı, tüm karmaşık karar mekanizmalarının temelini oluşturur.

Basit if Yapısı ve Söz Dizimi

if yapısı, program akışını bir koşula bağlamanın en temel yoludur.

Syntax: if (koşul) { kod bloğu };

Sadece tek bir koşul kontrol edilir.

Eğer koşul doğruysa içerideki kod çalışır, yanlışsa program akışına devam eder (hiçbir şey yapmaz).



if-else Yapısı: İki Yönlü Karar Mekanizması

if-else yapısı, koşulun hem doğru hem de yanlış olma durumlarına karşılık gelecek kesin kod yolları sunar.

Syntax: if (koşul) { doğruysa çalışır } else { yanlışsa çalışır };

Bu yapı, bir işlemin mutlaka iki sonuçtan birini üretmesi gerektiği durumlarda kullanılır.



Söz Dizimi Detayı: Küme Parantezlerinin Önemi

C# dilinde, if veya else bloğunda yalnızca tek bir komut çalıştırılacaksa küme parantezleri ({ }) isteğe bağlıdır.

Ancak birden fazla komutun bir blok içinde çalışması gerekiyorsa küme parantezleri zorunludur.

Akademik ve profesyonel yazılım geliştirmede, okunabilirliği artırmak ve hataları önlemek için tek komut olsa bile parantez kullanılması önerilir (Defensive Programming).



Karşılaştırma (İlişkisel) Operatörler – Bölüm 1

Karar yapıları içinde kullanılan koşullar, genellikle ilişkisel operatörlerle oluşturulur.

`==` (Eşittir): İki değerin birbirine eşit olup olmadığını kontrol eder. Sonuç Boolean'dır.

`!=` (Eşit değildir): İki değerin birbirinden farklı olup olmadığını kontrol eder.
Örnek: `if (kullaniciGirdisi == 'X')` veya `if (sonuc != 0)`.



Karşılaştırma (İlişkisel) Operatörler – Bölüm 2

Sayısal verileri kıyaslamak için kullanılırlar ve program akışını matematiksel durumlara göre yönlendirirler.

< (Küçüktür): Soldaki değer sağdakinden küçükse true döner.

> (Büyüktür): Soldaki değer sağdakinden büyükse true döner.

Bu operatörler özellikle sıralama, sınır kontrolü ve döngü sonlandırma koşullarında yaygındır.



Karşılaştırma (İlişkisel) Operatörler – Bölüm 3

Büyüklik veya küçüklük ile birlikte eşitlik durumunu da kontrol etmemizi sağlarlar.

$\leq$ (Küçük eşittir): Soldaki değer sağdakinden küçük veya eşitse true döner.

$\geq$ (Büyük eşittir): Soldaki değer sağdakinden büyük veya eşitse true döner.

Bu operatörler genellikle veri aralığı denetimlerinde veya dahil edici sınırların belirlenmesinde kullanılır.



Çoklu Karar Yapıları: if-else if Zinciri

Üç veya daha fazla alternatif durumun kontrol edilmesi gerektiğinde if-else if yapısı kullanılır.

Program, koşulları yukarıdan aşağıya doğru sırayla kontrol eder. İlk doğru koşul bulunduğunda, ilgili kod bloğu çalıştırılır ve tüm zincir atlanarak program akışına devam edilir.

Bu yapı, performans açısından verimlidir, çünkü gereksiz kontrolleri önler.



else if Kullanımının Avantajları

if-else if zinciri, bir dizi bağımsız if kullanmaktan daha etkilidir. Neden? Çünkü bir if doğru olduğunda, diğer koşulların kontrol edilmesine gerek kalmaz. Bu durum, özellikle çok sayıda koşulun olduğu kritik performans gerektiren sistemlerde önemlidir.



Son Alternatif: Sonlandırıcı else Bloğu

Bir if-else if zincirinin sonuna eklenen else bloğu, yukarıdaki koşulların hiçbirisi doğru değilse çalışır.

Bu blok, genellikle 'varsayılan durum' veya 'hata durumu' olarak kullanılır. else bloğunu kullanmak, tüm olası durumların program tarafından ele alındığından emin olmanızı sağlar (Complete Coverage).



Mantıksal Operatörler: Koşulları Birleştirme

Karmaşık iş mantığı, genellikle birden fazla basit koşulun aynı anda değerlendirilmesini gerektirir.

Mantıksal operatörler, Boolean ifadeleri birleştirerek yeni Boolean sonuçlar üretir.

Bu, programın daha ince ayrıntılı kararlar vermesini sağlar.



Mantıksal AND Operatörü (&&)

&& operatörü, birleşik koşulun sadece tüm alt koşullar true ise true dönmesini sağlar.

Örnek: `if (kullaniciAdiDogru && sifreDogru)`: Her iki bilgi de eşleşmeli. Gerçek hayattaki 've' bağlacı ile aynı mantıkta çalışır (Konjeksiyon).

Mantıksal OR Operatörü (||)

|| operatörü, birleşik koşulun alt koşullardan herhangi biri true ise true dönmesini sağlar.

Sadece tüm alt koşullar false ise false döner.

Gerçek hayattaki 'veya' bağlacı ile aynı mantıkta çalışır (Disjeksiyon).

Mantıksal NOT Operatörü (!)

! operatörü, bir Boolean ifadenin değerini tersine çevirir.
True ise false, false ise true yapar.
Örnek: if (!kullaniciGirisIzni): Kullanıcı giriş iznine sahip değilse çalışır.

Kısa Devre Değerlendirmesi (Short-Circuit Evaluation)

C#'taki && ve || gibi mantıksal operatörler kısa devre mantığıyla çalışır. && için, ilk koşul false ise, ikinci koşul kontrol edilmez (sonuç zaten false olacaktır).

|| için, ilk koşul true ise, ikinci koşul kontrol edilmez (sonuç zaten true olacaktır). Bu mekanizma performansı artırır ve potansiyel null referans hatalarını önlemede kullanılabilir.



İç İçe if Yapıları (Nested Ifs)

Bir if veya else bloğu içinde başka bir if yapısı kullanılmasıdır.

Bu, bir koşulun doğrulanmasından sonra, ikinci bir bağımlı koşulun kontrol edilmesini sağlar.

Okunabilirliği korumak için iç içe if yapılarının derinliği sınırlı tutulmalıdır; aksi takdirde if-else if veya mantıksal operatörler tercih edilmelidir.



Ternary Operatörü (Üçlü Operatör)

Ternary operatörü (`?:`), basit if-else atama işlemlerini tek bir satırda yapmamızı sağlar.

Syntax: `koşul ? doğru_ise_değer : yanlış_ise_değer;`

Değişken atama, kısa durum kontrolü veya basit değer seçimi için mükemmeldir.

Hata Yönetimi ve if Yapıları

Kullanıcıdan alınan verinin (input) beklenen formatta veya aralıkta olup olmadığını kontrol etmek için if yapıları temel araçlardır.

Örnek: `if (girilenYaş < 0 || girilenYaş > 150) { Hata mesajı göster };`

Bu, programın kararlılığını artırır ve beklenmeyen girdilerden kaynaklanan hataları önler.



Form Kontrolleri ve Karar Yapıları İlişkisi

Windows Forms (veya diğer UI teknolojileri) projelerinde, karar yapıları kullanıcı arayüzü (UI) kontrollerinden gelen verilere dayanır.

TextBox: Kullanıcıdan metin veya sayısal girdi almak için kullanılır.

Button: Bir olayı tetiklemek (Örn: 'Kontrol Et', 'Giriş Yap') ve if kod bloğunu çalıştırmak için kullanılır.

Label: İşlemin sonucunu veya durum bilgisini kullanıcıya göstermek için kullanılır.



Veri Dönüşümünün Zorunluluğu

TextBox gibi UI kontrollerinden alınan tüm veriler başlangıçta string (metin) formatındadır.

Karşılaştırma veya matematiksel işlemler yapabilmek için bu string verilerin sayısal tiplere (int, double vb.) dönüştürülmesi gerekir.

Bu dönüşüm olmadan, program sayısal bir mantıksal operatör kullanamaz.

Veri Dönüşüm Yöntemleri: Convert.ToInt32()

Convert.ToInt32() metodu, herhangi bir temel veri tipini (string dahil) 32 bit imzalı tamsayıya (int) dönüştürmeye çalışır.

Eğer dönüştürülecek string, geçerli bir sayı içermiyorsa, bu metod bir 'FormatException' hatası fırlatır.

Avantajı: null değerleri 0'a dönüştürebilmesidir.

Veri Dönüşüm Yöntemleri: int.Parse()

int.Parse() metodu, yalnızca string ifadeleri tamsayılara dönüştürmek için kullanılır.

Bu metod, eğer string boş (null) ise veya geçerli bir sayı formatında değilse hata (Exception) fırlatır.

Genellikle, kesinlikle sayısal bir string beklendiği durumlarda tercih edilir.

Gelişmiş Dönüşüm: TryParse Kavramı

Üniversite düzeyinde, hatalı kullanıcı girdisine karşı programın çökmemesi için TryParse metotları önerilir.

int.TryParse() metodu, dönüşüm başarılı olursa true döner ve sonucu 'out' parametresi aracılığıyla sağlar.

Dönüşüm başarısız olursa false döner ve hata fırlatılmaz.

if (int.TryParse(textBox.Text, out sayi)) {...} yapısı idealdir.

Proje 1: Sayı Tahmin Oyunu Tanıtımı

Bu proje, if-else if zincirini ve ilişkisel operatörleri uygulamalı olarak öğrenmeyi amaçlar.

Temel Mantık: Programın tuttuğu gizli bir sayıyı, kullanıcının tahminleri ile karşılaştırmak ve yol göstermektir.



Proje 1 Arayüz (UI) Bileşenleri

Arayüz için gereken minimum bileşenler şunlardır:

1 adet TextBox: Kullanıcının tahminini girmesi için.

1 adet Button: Tahmini kontrol etme eylemini başlatmak için ('Tahmin Et' yazılı).

1 adet Label: Tahmin sonucunu (Daha büyük, Daha küçük, Bildiniz) göstermek için.

Proje 1: Sayı Saklama Mekanizması

Gizli sayıyı üretmek için C#'ın Random sınıfı kullanılır.
Sayı, Form yüklendiğinde (Form_Load) 1 ile 100 arasında rastgele tutulur.
Bu sayının, Button Click olayında da erişilebilir olması gerektiğinden, global (form seviyesinde) bir değişkende saklanmalıdır.

Proje 1: Button Click Olayı – Veri İşleme

Kullanıcı 'Tahmin Et' butonuna bastığında çalışacak ilk adımlar:

1. TextBox'tan gelen string değer okunur.
2. Okunan değer, sayısal karşılaştırma için 'int.Parse()' veya 'Convert.ToInt32()' ile tamsayıya çevrilir.
3. Girilen sayı, global olarak saklanan tutulan sayı ile karşılaştırılmaya hazırdır.

Proje 1: Button Click Olayı – Eşitlik Kontrolü

Karar yapısının ilk koşulu, kullanıcının sayıyı doğru tahmin edip etmediğini kontrol eder.

if (girilenSayi == tutulanSayi): Bu, doğrudan eşitlik operatörünü kullanır. Eğer doğruysa, Label kontrolüne 'BİLDİNİZ!' mesajı yazılır.



Proje 1: Button Click Olayı – 'Daha Büyük' Koşulu

Eşitlik koşulu yanlışsa (sayı bilinmediyse), if-else if zincirinde ikinci kontrol devreye girer.

else if (girilenSayı < tutulanSayı): Kullanıcının tahmini, programın tuttuğu sayıdan küçükse.

Bu durumda Label'a 'Daha büyük bir sayı girin.' mesajı yazılır.

Proje 1: Button Click Olayı – Son else Bloğu

Eğer sayı ne eşitse ne de küçükse, mantıksal olarak geriye sadece tek bir olasılık kalmıştır: Tahmin edilen sayı büyüktür.

else: Bu blok çalışır.

Label'a 'Daha küçük bir sayı girin.' mesajı yazılır.

Proje 1: if-else if Akışının Özeti

Bu proje, if-else if-else yapısının doğru hiyerarşiyi kurmadaki gücünü gösterir.

1. Eşit mi? (Kazanma)

2. Küçük mü? (Yönlendirme Yukarı)

3. Değilse, büyüktür. (Yönlendirme Aşağı)

Bu üç aşamalı yapı, tüm olası kıyaslama durumlarını kapsar.

Proje 2: Kullanıcı Giriş Paneli Tanıtımı

Bu proje, kullanıcı adı ve şifre gibi iki ayrı verinin aynı anda doğru olmasının kontrol edilmesini gerektirir.

Temel Mantık: Bir sisteme erişim için iki farklı koşulun da zorunlu olduğu güvenlik senaryolarını simüle eder.



Proje 2 Arayüz (UI) Bileşenleri

Arayüz için gereken bileşenler:

2 adet TextBox: Biri Kullanıcı Adı için, diğeri Şifre için.

Şifre TextBox'ının kritik bir özelliği: PasswordChar özelliği '*' olarak ayarlanmalıdır. Bu, girilen şifrenin gizlenmesini sağlar.

1 adet Button: 'Giriş Yap' eylemini tetikler.

1 adet Label: Giriş sonucunu göstermek için ('Başarılı' veya 'Hata').

Proje 2: Şifre Gizleme Güvenliği

Şifre alanındaki karakterlerin '*' veya benzeri bir karakterle maskelenmesi, temel bir kullanıcı güvenliği uygulamasıdır. Bu özellik, ekrandaki şifrenin üçüncü gözler tarafından okunmasını engeller. Ancak bu özellik, şifrenin hafızada yine düz metin olarak saklandığını unutmamalıdır (Gerçek şifre güvenliği hashing gerektirir).



Proje 2: Button Click Olayı – Veri Alma

Kullanıcı 'Giriş Yap' butonuna tıkladığında, TextBox'lardan kullanıcı adı (kullaniciAdiText) ve şifre (sifreText) string olarak okunur. Bu projede sayısal dönüşüm gerekmez, çünkü karşılaştırma string (metin) düzeyinde yapılacaktır.

Proje 2: Mantıksal && Uygulaması

Girişin başarılı olması için hem kullanıcı adının hem de şifrenin önceden tanımlı değerlerle eşleşmesi gerekir.

Tanımlı Değerler: Kullanıcı adı için 'admin', Şifre için '12345'.

if (kullaniciAdiText == 'admin' && sifreText == '12345'): Bu ifade, iki koşulun da doğru olmasını bekler.



Proje 2: Başarılı Giriş Durumu

Eğer && operatörü true sonucunu döndürürse (yani her iki koşul da doğruysa), Label'a 'Giriş Başarılı' mesajı yazılır.
Bu, kullanıcıya sisteme erişim yetkisi verildiğini gösterir.



Proje 2: Hata Durumu Yönetimi

if koşulunun yanlış olması, ya kullanıcı adının, ya şifrenin ya da her ikisinin de yanlış olduğu anlamına gelir.

else bloğu çalışır:

Label'a 'Hata! Bilgilerinizi kontrol edin.' mesajı yazılır.

Bu genel hata mesajı, kötü niyetli denemelerde hangi bilginin yanlış olduğunu ifşa etmemek adına güvenlik için önemlidir.

String Karşılaştırmalarında Dikkat Edilmesi Gerekenler

C# ve birçok programlama dili, string karşılaştırmalarında varsayılan olarak büyük/küçük harf duyarlıdır (Case Sensitive).

'Admin' ve 'admin' birbirinden farklı kabul edilir.

Eğer harf duyarlılığının önemsiz olduğu bir karşılaştırma isteniyorsa, 'String.Equals()' veya 'ToLower()/ToUpper()' metotları kullanılmalıdır (Örn: 'kullaniciAdi.ToLower() == 'admin)').

if-else Kullanımında Akademik Yaklaşımlar

Karmaşık iç içe if'ler yerine, 'Erken Çıkış' prensibi önerilir.
Bu prensipte, hata veya istisna durumları fonksiyonun veya metodun başında kontrol edilir.
Eğer bir koşul sağlanmıyorsa hemen metoddan çıkılır (return veya throw ile).
Bu, ana iş mantığının daha düz ve okunabilir kalmasını sağlar.

Kodu Tekrarlamama Prensibi (DRY)

Eğer aynı koşul veya hesaplama birden fazla if bloğunda kullanılıyorsa, bu ifade bir Boolean değişkene atanmalıdır.

Örn: `bool yetkiliMi = (yaş >= 18 && ehliyetVar);`

Bu, hem kod tekrarını önler hem de mantıksal ifadenin tek bir yerden yönetilmesini kolaylaştırır.

Karar Yapılarına Alternatifler: Switch-Case

if-else if zincirleri, özellikle tek bir değişkenin birden fazla sabit değere karşı kontrol edilmesi gerektiğinde (sadece == operatörü) uzayabilir. switch-case yapısı, bu tür senaryolarda daha temiz ve okunabilir bir alternatif sunar. switch yapısı, genellikle performans açısından da if-else if zincirlerine göre avantajlıdır (derleyici optimizasyonu).

C# Pattern Matching (Model Eşleştirme)

Modern C# (C# 7.0 ve sonrası), if-else yapısının yeteneklerini artıran 'Pattern Matching' özelliklerini tanıtmıştır.

Bu özellik, sadece değeri değil, aynı zamanda veri tipini de kontrol etmeyi, null kontrolü yapmayı ve nesne özelliklerini eşleştirmeyi if-else içinde veya switch ifadelerinde çok daha zarif hale getirir.



if Bloklarının Bakım Zorluğu (Maintainability)

Çok fazla iç içe if bloğu (Deep Nesting), 'Spagetti Kodu'na yol açarak programın anlaşılmasını ve test edilmesini zorlaştırır.

Akademik yazılım mühendisliği prensipleri, karar yapılarını mümkün olduğunca 'düzleştirmeyi' (Flatness) önerir.

Bunun için polimorfizm (çok biçimlilik) veya komut (Command) desenleri gibi nesne yönelimli teknikler kullanılabilir.

Her bir if ve else if bloğu, programın farklı bir 'yolunu' (Path) temsil eder. Kaliteli bir yazılım geliştirmek için, bu yolların her birinin ayrı ayrı test edilmesi gerekir (Path Coverage). Karmaşık if-else if yapıları, test edilecek senaryo sayısını (Cyclomatic Complexity) hızla artırır.



Özet: if-else Temel Prensipleri

if ve else, C# program akışını kontrol eden temel mekanizmalardır. Karar verme, ilişkisel ve mantıksal operatörlere dayanır. Projelerde kullanıcı girdisini işlemek için veri dönüşümü zorunludur (int.Parse(), Convert.ToInt32()). if-else if zinciri, birden çok durumu sırayla ve verimli bir şekilde ele alır.

İpuçları ve En İyi Uygulamalar

1. Koşulları olabildiğince basit tutun ve karmaşık mantığı değişkenlere ayırın.
2. Küme parantezlerini her zaman kullanın, tek komut olsa bile.
3. Karşılaştırmalarda string verileri kontrol ederken büyük/küçük harf duyarlılığını dikkate alın.
4. Kullanıcı girdisi alırken `int.TryParse` gibi güvenli dönüşüm metotlarını tercih edin.

C# Karar Yapılarına Giriş

Süleyman **EZDEMİR**

suleyman.ezdemir@giresun.edu.tr

